

발표 준비 완전 학습 노트

코딩 에이전트 다음 행동 예측 · Team MOOD

용어 사전부터 예상 질문 대응까지, 발표자가 혼자 공부할 수 있게 정리한 자료

최종 성적 **Private Macro-F1 0.7967670316** · 제출물 **g4_c15.zip** · LightGBM 1 + mmBERT 4 게이트 앙상블

공부 순서 — Part 0 → Part 1 → **Part 2(용어 사전, 가장 중요)** → Part 3 → Part 4 → Part 5(슬라이드별) → Part 6(Q&A) → Part 7(암기 카드) → Part 8(퀴즈)

Part 2를 이해하지 못한 채 Part 5로 건너뛰면 대본을 외우게 될 뿐입니다. **용어 사전에 30분을 쓰는 게 대본을 세 번 읽는 것보다 낫습니다.**

Part 8의 퀴즈는 답을 가리고 **소리 내어** 답해 보세요. 막히는 항목이 그날 공부할 부분입니다.

목차

Part 0 한 장으로 보는 전체 그림

Part 1 대회 자체를 이해하기

Part 2 용어 사전 ★ 가장 중요 ★

Part 3 우리 시스템이 실제로 돌아가는 순서

Part 4 우리가 실패한 것들 (슬라이드 13 — 신뢰성 어필 구간)

Part 5 슬라이드별 "무엇을 이해하고 올라가야 하는가"

Part 6 예상 질문과 답변 (Q&A 대비)

Part 7 숫자 암기 카드

Part 8 자가 점검 퀴즈

Part 9 발표 직전 체크리스트

발표 준비 완전 학습 노트

이 문서의 목적: 발표자가 용어를 전혀 모르는 상태에서 출발해, 10분 발표와 Q&A를 스스로의 언어로 소화할 수 있게 만드는 것.

공부 순서: Part 0 → Part 1 → **Part 2(용어사전, 가장 중요)** → Part 3 → Part 4 → Part 5(슬라이드별) → Part 6(Q&A) → Part 7(암기카드) → Part 8(퀴즈로 자가진단)

Part 2를 이해하지 못한 채 Part 5로 건너뛰면 대본을 외우게 될 뿐입니다. **용어사전에 30분을 쓰는 게 대본을 3번 읽는 것보다 낫습니다.**

Part 0. 한 장으로 보는 전체 그림

0-1. 세 문장 요약

- 1. 문제: AI 코딩 도우미(예: Claude Code, Cursor)가 사용자와 대화하다가 **바로 다음에 무슨 행동을 할지**(파일 읽기? 테스트 실행? 웹 검색?)를 14가지 중 하나로 맞히는 문제.
- 2. 우리가 한 일: 대회가 준 기본 예제(점수 0.43)에서 시작해, 데이터를 뜯어보고 → 검증 체계를 세우고 → 여러 모델을 조합해서 **0.797**까지 올렸다.
- 3. 핵심 무기 2가지: ① ID토큰 직렬화(세션 번호를 AI 모델에게 글자로 알려주기) ② 게이트 앙상블(어려운 문제에만 추가 모델을 투입하기).

0-2. 우리 시스템을 일상 비유로

회사에 판정 위원 5명이 있습니다.

- 위원 1 (LightGBM): 표·통계에 강한 회계사 타입. 발언권 10%.
- 위원 2 (mmBERT s77): 글을 잘 읽는 사람. 발언권 45%. 자료를 "A방식"으로 정리해 봄.
- 위원 3 (mmBERT s7): 역시 글을 잘 읽지만, 자료를 "B방식"으로 정리해 봄. 발언권 25%.
- 위원 2와 3이 같은 답을 내면 → 회의 끝. (전체의 약 95%)
- 둘이 다른 답을 내면(약 5%) → 위원 4(s99)를 심판으로 호출 (발언권 20%).
- 위원 4가 답을 냈는데, 위원 5(s123)까지 위원 4와 같은 답이고 + 원래 둘 중 한 명도 그 답을 지지하면 → 위원 5도 추가 투입 (발언권 15%).

이렇게 모은 의견을 **가중 평균**해서 가장 점수 높은 행동 하나를 최종 답으로 냅니다.

이 비유가 머리에 남으면 발표의 절반은 끝난 겁니다.

0-3. 점수의 성장 궤적 (이 숫자 흐름만은 외우기)

0.43	←	대회가 준 기본 예제 (프롬프트 글자만 사용)	
0.61	←	행동 이력 + 메타정보를 글자로 붙임	[데이터 분석의 승리]
0.74	←	LightGBM + 트랜스포머 블렌드	[모델 조합의 시작]
0.77	←	백본을 mmBERT로 교체	[최신 모델의 힘]
0.790	←	ID토큰 직렬화	[핵심 기여 ①, +0.011]
0.794	←	서로 다른 방식으로 정리한 2모델 앙상블	
0.797	←	게이트 앙상블	[핵심 기여 ②]

Part 1. 대회 자체를 이해하기

1-1. 무엇을 맞히는 문제인가

AI 코딩 에이전트가 개발자와 대화하며 일하는 한 시점의 스냅샷을 주고, 그 다음에 에이전트가 취할 행동을 맞히는 문제입니다.

우리가 받는 입력 3가지:

항목	뜻	예시
<code>current_prompt</code>	방금 사용자가 한 말	"테스트 좀 돌려봐"
<code>history</code>	지금까지의 대화·행동 기록 (0~12개)	<code>read_file(ok)</code> → <code>grep_search(17건 발견)</code>
<code>session_meta</code>	작업 환경 정보	턴 번호 6, CI 실패, 미커밋 변경 있음, 열린 파일 3개

맞혀야 하는 출력: 14개 행동 중 하나

묶음	클래스	뜻
탐색/ 검색	<code>read_file</code> / <code>grep_search</code> / <code>list_directory</code> / <code>glob_pattern</code>	파일 읽기 / 패턴 검색 / 폴더 목록 / 파일명 패턴 검색
파일 수정	<code>edit_file</code> / <code>write_file</code> / <code>apply_patch</code>	기존 파일 수정 / 새 파일 작성 / 패치 적용
실행/ 검사	<code>run_bash</code> / <code>run_tests</code> / <code>lint_or_typecheck</code>	셸 명령 / 테스트 / 문법·타입 검사
대화/ 기타	<code>ask_user</code> / <code>plan_task</code> / <code>web_search</code> / <code>respond_only</code>	되묻기 / 계획 수립 / 웹 검색 / 말로만 답하기

발표 중 클래스 이름을 물어보면 "탐색 4개, 수정 3개, 실행 3개, 대화 4개"로 묶어서 답하면 됩니다.

1-2. 데이터 규모

- 학습 데이터: 70,000건 (정답 포함) — 우리가 자유롭게 볼 수 있음
- 평가 데이터: 30,000건 (정답 없음, 우리는 볼 수 없음) — 대회 서버 안에만 존재
- 우리가 받은 `test.jsonl` 은 형식 확인용 5건뿐

1-3. 채점 방식 — Macro-F1

이게 왜 중요한지 이해해야 발표 전체가 이해됩니다.

- **F1 점수**: "정밀도(Precision)"와 "재현율(Recall)"의 조화 평균. 0~1 사이 값. 높을수록 좋음.
- **Macro(매크로)**: 14개 클래스 각각의 F1을 구한 뒤 **똑같은 무게로 평균**.

여기서 결정적인 함의:

우리 데이터는 가장 많은 클래스(`edit_file`, 11,171건)가 가장 적은 클래스(`web_search`, 1,273건)의 **9배**입니다. 만약 채점이 "전체 정답률"이었다면 `web_search`는 무시해도 점수에 거의 영향이 없습니다. 그런데 Macro-F1은 `web_search` 하나의 F1이 `edit_file`의 F1과 **정확히 같은 무게**로 반영됩니다.

한 문장으로: "적게 나오는 행동을 못 맞히면 순위가 무너진다." 이게 우리가 소수 클래스에 신경 쓴 이유이고, LightGBM 풀백에서 `web_search` 확률에만 **1.75배 가중**을 준 이유입니다.

1-4. 제출 방식 — CSV가 아니라 "코드"를 낸다 (이 대회만의 특이점)

보통의 AI 대회는 "정답 파일(CSV)"을 제출합니다. 이 대회는 다릅니다.

```
제출.zip
├─ model/           ← 미리 학습해 둔 모델 파일들
├─ script.py        ← 채점 서버가 그대로 실행하는 추론 코드 (파일명 고정)
└─ requirements.txt ← 필요한 라이브러리 목록
```

서버가 zip을 풀고 → `python script.py`를 실행 → 우리 코드가 `output/submission.csv`를 만들어야 채점됩니다.

여기서 나오는 제약이 곧 우리 설계를 규정했습니다:

제약	수치	우리에게 생긴 숙제
GPU	NVIDIA T4 (VRAM 16GB)	트랜스포머 사용 가능! (다행)
CPU	3 vCPU (약함)	무거운 CPU 전처리 금지
추론 시간	10분 이내	초과하면 제출 실패 + 제출 횟수 차감
zip 용량	1GB 이하	모델 5개를 다 넣기엔 원래 불가능 → 어휘 축소로 해결
인터넷	차단	사전학습 모델을 인터넷에서 못 받음 → zip에 넣어야 함
제출 횟수	제한 있음	아무거나 던져볼 수 없음 → 실험 설계가 중요

1-5. Public / Private 점수 — 이 대회만의 중요한 특징

- 보통 대회: 리더보드(Public)는 평가 데이터의 일부만 채점하고, 최종 순위(Private)는 나머지로 매김. 그래서 "리더보드에 맞춰 튜닝하면 최종에서 망한다(=Public 과적합)"는 게 상식입니다.
- 이 대회: **Public = 전체 테스트 100% 채점**, Private = 마감 시점의 Public 그대로.

- **함의**: 리더보드 점수가 곧 최종 점수. 따라서 **리더보드에 맞춰 미세조정하는 것이 "과적합"이 아니라 "진짜 목적함수를 직접 측정하는 것"**입니다.

Q&A에서 "LB 프로빙은 과적합 아니냐"는 질문이 나오면 이걸로 답합니다. (Part 6 참조)

Part 2. 용어 사전 ★ 가장 중요 ★

레벨 1 — 머신러닝 기초 용어

분류 / 다중 분류 (Classification / Multi-class)

정답이 "숫자"가 아니라 "카테고리"인 문제. 14개 중 하나를 고르므로 **14-클래스 다중 분류**.

클래스(class) — 맞혀야 할 카테고리 하나하나. 여기서 `read_file` 같은 행동 14개.

라벨(label) — 정답. `train_labels.csv` 에 들어 있는 그 값.

피처(feature, 특징) — 모델에게 주는 입력 재료 하나하나. 예: "턴 번호", "CI가 실패했는가", "프롬프트 글자 수". 모델은 피처를 보고 답을 추측합니다.

피처 엔지니어링 — 원본 데이터에서 "모델이 알아듣기 좋은 피처"를 사람이 설계해 만드는 작업. 우리 LightGBM은 텍스트 14 + 구조 49 + 표적 11 + ID 13 = 87개 피처를 씁니다.

학습(train) / 검증(validation) / 평가(test)

- 학습: 모델이 정답을 보며 배우는 데이터
- 검증: 배운 모델이 잘하는지 우리가 **로컬에서** 재보는 데이터 (정답 있음)
- 평가: 대회 서버가 채점하는 데이터 (정답 모름)

과적합(overfitting) — 모델이나 설정이 "내가 본 데이터에만" 잘 맞게 되어버리는 현상. 검증 점수는 오르는데 실제 점수는 떨어지는 상황. **이 발표의 큰 줄기 중 하나가 "우리는 과적합을 3번 조기에 잡아냈다"입니다.**

하이퍼파라미터 — 모델이 스스로 배우는 게 아니라 **사람이 정해주는** 설정값. 예: 학습률, 에폭 수, 앙상블 가중치 0.45.

시드(seed) — 난수 초기값. 같은 시드면 같은 결과가 나와 **재현 가능**해집니다. 우리는 트랜스포머를 시드 7, 77, 99, 123으로 4번 학습했습니다. 같은 데이터라도 시드가 다르면 **조금씩 다른 모델**이 되고, 이 "다름"이 앙상블의 재료입니다.

에폭(epoch) — 전체 학습 데이터를 처음부터 끝까지 한 바퀴 도는 것. 우리는 3에폭 학습. (6에폭이 정점이던 시절도 있었으나 mmBERT 백본에서는 3에폭이 최적이었습니다.)

배치(batch) — 한 번에 묶어서 처리하는 샘플 개수. 우리는 학습 시 16개씩.

학습률(learning rate, lr) — 한 번에 얼마나 크게 배울지. 우리는 $3e-5$ ($=0.00003$).

레벨 2 — 우리가 쓴 모델들

TF-IDF + 로지스틱 회귀 (베이스라인, 0.43)

- **TF-IDF**: 문장을 숫자 벡터로 바꾸는 고전 기법. "이 문서에 자주 나오면서(TF), 다른 문서엔 드물게 나오는 (IDF) 단어일수록 중요하다"는 원리. 단어를 **의미가 아니라 개수로만** 봅니다. "테스트"와 "test"가 다른

단어로 취급됩니다.

- **로지스틱 회귀**: 가장 단순한 분류 모델. 각 단어에 가중치를 곱해 더하는 수준.

발표 포인트: 대회가 준 베이스라인은 `current_prompt` 글자 하나만 썼습니다. history도 session_meta도 통째로 버리고 있었습니다. 그게 0.43의 이유입니다.

LightGBM (그래디언트 부스팅)

- **결정 트리(decision tree)**: "CI가 실패했나? → 예 → 직전 행동이 edit_file인가? → 예 → run_tests" 같은 스무고개 구조.
- **그래디언트 부스팅**: 트리 하나를 만들고, 그게 틀린 부분만 집중해서 다음 트리를 만들고, 이걸 수백 번 반복해 합치는 방식. 우리는 트리 500그루.
- **LightGBM**: 그래디언트 부스팅의 빠른 구현체 (Microsoft).
- **왜 좋은가**: 표 형태의 숫자-카테고리 데이터, 그리고 **피쳐끼리의 조합** ("CI 실패 그리고 직전이 edit_file")을 잡는 데 강합니다.

스태킹(stack) / OOF

- **스태킹**: 모델 A의 예측 결과를 모델 B의 입력 피쳐로 쓰는 2층 구조. 우리는 TF-IDF 텍스트 모델의 14개 클래스 확률을 LightGBM의 피쳐로 넣었습니다.
- **OOF (Out-Of-Fold)**: 스태킹의 필수 안전장치. 자기가 학습한 데이터에 대해 예측하면 "이미 정답을 본" 부풀려진 확률이 나옵니다. 그래서 데이터를 여러 조각(fold)으로 나눠, **각 조각을 예측할 땐 그 조각을 빼고 학습한 모델을 씁니다.** 그렇게 만든 예측이 OOF 예측.

누수(leakage)와 직결되는 개념입니다. 나중에 "학습형 메타 게이트가 실패한 이유"가 바로 OOF를 썼는데도 트랜스포머 확률이 전체 학습본에서 나와 누수가 남았기 때문입니다.

트랜스포머 / BERT / 사전학습 / 파인튜닝 ★

이 대회 후반의 주인공입니다.

- **트랜스포머(Transformer)**: 2017년 등장한 신경망 구조. 문장 안의 단어들이 **서로를 참조(attention)** 하면서 문맥을 이해합니다. GPT, BERT 전부 이 구조입니다.
- **BERT**: 트랜스포머로 만든 "문장 이해 전용" 모델 계열. 글을 생성하지 않고 **읽고 분류**하는 데 특화.
- **사전학습(pre-training)**: 인터넷의 방대한 텍스트로 언어 자체를 미리 배워둔 상태. 이 결과물을 **백본(backbone)** 또는 사전학습 가중치라고 부릅니다.
- **파인튜닝(fine-tuning)**: 사전학습된 백본을 가져와, **우리 문제 데이터(7만 건)로 추가 학습**시켜 우리 문제 전용 모델로 만드는 것. 우리가 한 일이 이겁니다.

비유: 사전학습 = 국어를 배운 사람. 파인튜닝 = 그 사람에게 우리 회사 업무를 가르치는 것. 국어부터 가르치는 것보다 압도적으로 빠르고 잘합니다.

- **mmBERT**: 2025년 나온 **다국어** BERT 계열 백본 (`jhu-clsp/mmBERT-base`). 우리 데이터가 한국어/영어/혼용이라 다국어 모델이 필요했습니다.
- **XLM-R (XLM-RoBERTa)**: 다국어 모델의 기존 표준. 우리가 처음에 쓰던 것. **mmBERT로 바꾸자 단독 +0.021** — 단일 변경으로는 최대 개선이었습니다.
- **mDeBERTa**: 시도했으나 학습이 깨져서(loss가 NaN) 포기한 후보.

토큰나이저 / 서브워드 / 어휘 / max_len

- **토큰나이저(tokenizer)**: 문장을 모델이 먹을 수 있는 조각(토큰)으로 자르는 도구.
- **서브워드(subword)**: 단어보다 작은 조각. "테스트하기"를 "테스트"+"하기"로 자르는 식. 처음 보는 단어도 조각 조합으로 표현할 수 있어 다국어에 유리합니다.
- **어휘(vocabulary)**: 토큰나이저가 아는 토큰의 전체 목록. mmBERT는 **약 256,000개**. 다국어라 세계 모든 언어의 토큰을 갖고 있습니다.
- **max_len**: 모델이 한 번에 읽을 수 있는 최대 토큰 수. 우리는 **512**. 넘으면 잘립니다. 그래서 **최근 대화를 앞에 배치**해 잘려도 오래된 것부터 잘리게 했습니다.

확률 / softmax / argmax / 온도(temperature)

- 모델의 출력은 "정답 하나"가 아니라 **14개 클래스 각각의 확률**입니다. 예: `[read_file 0.6, edit_file 0.3, run_tests 0.05, ...]` (합 = 1)
- **argmax**: 그중 가장 큰 것을 고르는 연산. 최종 예측을 만드는 마지막 단계.
- **온도(temperature) 보정**: 확률 분포를 더 **뾰족하게** 또는 더 **평평하게** 만드는 조정. 우리는 확률에 **0.28** 제곱을 한 뒤 다시 합이 1이 되게 정규화합니다.
- 예: $0.9 \rightarrow 0.9^{0.28} \approx 0.97 / 0.1 \rightarrow 0.1^{0.28} \approx 0.53 \rightarrow$ **격차가 줄어듦 = 평평해짐**
- **왜?** 한 모델이 지나치게 자신만만하면 앙상블에서 그 모델이 독주합니다. 평평하게 만들면 다른 모델의 2:3순위 의견까지 반영되어 조합의 효과가 커집니다.

레벨 3 — 우리가 만든 기법들 ★★ 발표의 핵심 ★★

① 직렬화(serialization)

정의: 흩어져 있는 입력(프롬프트 + 이력 + 메타)을 **트랜스포머가 읽을 수 있는 하나의 긴 문자열**로 합치는 작업.

트랜스포머는 "텍스트 한 덩어리"만 먹습니다. 그런데 우리 입력은 텍스트 + 목록 + 숫자가 섞여 있습니다. 그래서 전부 글자로 바꿔 이어붙였습니다.

기본 형태:

```
[프롬프트] || [메타 토큰] || [최근 순 대화 이력]
```

실제 예:

```
"테스트 돌려봐" || CI_failed TURN_5 DIRTY_True TIER_pro PREF_ko OPEN_2 LANG_py FILES app/main.py
|| A: edit_file(app/main.py) -> ok | U: "여기 고쳐줘" | A: read_file(app/main.py) -> ok
```

- 이력을 **최근 것부터 역순**으로 넣습니다 → 512토큰을 넘겨서 잘릴 때 **오래된 것부터** 잘리도록.
- 행동의 인자(파일 경로 등)는 40자로 잘라서 포함 → 경로/패턴이 탐색 계열 4개를 구분하는 단서.

② ID토큰 직렬화 (핵심 기여 1, LB +0.011) ★★★

발견: LightGBM의 피처 중요도를 뽑아 봤더니, **가장 강한 피처가 "세션 번호"**였습니다.

(sess_sim_20260522_028750-step_02 의 028750 과 02)

왜 세션 번호가 예측에 도움이 되는가? 데이터가 시뮬레이션으로 생성되었기 때문에, 비슷한 번호대의 세션이 비슷한 시나리오 배치에서 나왔을 가능성이 큼니다. 즉 번호 자체가 **어떤 종류의 세션인지에 대한 힌트**를 담고 있습니다.

문제: LightGBM은 이 신호를 쓰는데, **트랜스포머는 전혀 못 보고 있었습니다.** 직렬화 문자열에 id가 안 들어 있었으니까요.

해결: 세션 번호를 **서브워드 친화적인 토큰으로 쪼개서** 직렬화 맨 앞에 주입:

```
SID 28750 STEP 2 M256_98 M100_50 M64_34 M16_2 | (기존 직렬화...)
```

- **SID** : 세션 번호, **STEP** : 몇 번째 스텝인지
- **M256_** , **M100_** , **M64_** , **M16_** : 세션 번호를 256/100/64/16으로 나눈 **나머지**

왜 나머지(modulo)를 넣는가? 숫자 "28750"을 그대로 주면 토큰라이저가 이상하게 자르고, 모델은 큰 수를 잘 다루지 못합니다. 나머지로 잘게 쪼개면 **작은 정수 여러 개**가 되어 모델이 훨씬 잘 학습합니다. 여러 배수로 나눈 이유는 서로 다른 주기의 패턴을 동시에 노출하기 위해서입니다.

효과 3가지 (발표에서 반드시 말할 것):

1. 리더보드 **+0.011** — 단일 기법 최대 기여
2. 트랜스포머가 LightGBM의 구조 신호를 **흡수** → LightGBM 가중치가 0.4 → 0.2 → **최종 0.10**까지 하락
3. **부산물:** ID토큰만 넣은 버전(v2)과 신호토큰까지 넣은 버전(v3)이 **서로 다른 오류**를 냄 → 이게 다음 기여 (양상불)의 토대가 됨

③ v2 / v3 직렬화의 차이 (정확히 알아둘 것)

발표 자료에서는 "v2 = ID토큰 / v3 = 신호토큰"으로 줄여 부르지만, 정확히는:

버전	구성
v2	ID토큰 + 기본 직렬화
v3	ID토큰 + 구조 신호 토큰 + 기본 직렬화

구조 신호 토큰(v3에만 추가되는 부분) — LightGBM이 강하게 쓰던 신호를 명시적 토큰으로:

```
CIF_failed DIRTY_1 NOPEN_2 EXT_py EXT_sql NACT_4 RES_fail PREV_read_file PREV_edit_file
SEQ_read_file>edit_file
```

- RES_fail/RES_zero/RES_ok : 직전 행동의 결과가 실패인지, 0건 검색인지, 성공인지
- SEQ_A>B : 직전 두 행동의 전이 패턴
- NACT_ : 지금까지 행동 개수, EXT_ : 열린 파일 확장자

면접식 질문 대비: "그럼 v3가 정보가 더 많으니 항상 더 좋은 것 아닌가?" → 아닙니다. 명시적 신호를 주면 모델이 그 신호에 의존하게 되어, 원문 텍스트에서 찾아냈을 미묘한 단서를 덜 봅니다. 실제로 두 버전은 성능은 비슷한데 틀리는 문제가 다릅니다. 앙상블에서는 이 "다르게 틀림"이 성능보다 더 중요한 자산입니다.

④ 앙상블(ensemble)과 "다양성"

- 앙상블: 여러 모델의 의견을 합쳐 하나의 답을 내는 것.
- 소프트 보팅: 각 모델의 확률을 가중 평균한 뒤 argmax. 우리 방식이 이겁니다. (하드 보팅 = 각자 낸 답을 다수결로 세는 것)
- 다양성(diversity): 앙상블이 효과를 내려면 모델들이 서로 다른 실수를 해야 합니다. 똑같이 틀리는 모델 5개를 모아봐야 똑같은 답만 나옵니다.

우리가 실측한 다양성 수치 (발표 슬라이드 7의 근거):

- 같은 계열(같은 백본·같은 직렬화, 시드만 다름) 모델끼리 이견율 7.3%
- 다른 백본·다른 직렬화 모델끼리 이견율 13.5%
- → 결론: 다양성은 시드가 아니라 표현·아키텍처를 바꿔야 나온다.

⑤ 모델 수프(model soup) — 우리가 기각한 것

- 정의: 같은 구조의 모델 여러 개의 가중치를 직접 평균해서 모델 하나로 합치는 기법. (확률을 평균하는 앙상블과 다름 — 이건 모델의 파라미터 자체를 평균)
- 결과: 0.7645로 붕괴했습니다.
- 이유: 시드가 다르면 학습이 서로 다른 골짜기(basin)로 수렴합니다. 특히 분류 헤드(마지막 층)가 완전히 다른 좌표계를 쓰게 되어, 평균하면 둘 다 아닌 무의미한 지점이 됩니다.

비유: 서울에서 부산 가는 길과 인천 가는 길의 중간 지점은 어디에도 도착하지 못하는 곳입니다.

⑥ 선택적 게이트 앙상블 (핵심 기여 2) ★★★

아이디어 한 줄: 모든 문제에 모든 모델을 쓰지 않는다. 어려운 문제에만 추가 의견을 받는다.

동작 (실제 코드 기준):

```
1단계 기본 2모델(s77=v2, s7=v3)이 30,000건 전부 예측
↓
2단계 두 모델의 argmax가 같은가?
├─ 같음 (약 95%) → 그대로 진행. 추가 모델 호출 없음.
└─ 다름 (약 5%) → 그 행만 골라 3번째 모델(s99, judge)에 투입, 가중치 0.20
↓
3단계 4번째 모델(s123, consensus)을 그 행들에만 투입.
단, 아래 조건을 모두 만족할 때만 의견을 반영 (가중치 0.15):
  · s123의 답 == s99(judge)의 답      ← 심판과 같은 의견이고
  · 그리고 그 답이 기본 2모델 중 한쪽의 답과도 일치 ← 원래 후보 안에 있고
조건을 못 채우면 s123의 의견은 버림.
↓
4단계 모인 확률들을 가중 평균 → argmax → 최종 답
```

용어 정리:

- **게이트(gate):** "문", 즉 조건을 만족할 때만 통과시키는 스위치. `disagreement` 게이트 = 기본 두 모델이 불일치할 때만 열림. `disagreement_consensus` 게이트 = 불일치 + 합의 조건까지 만족할 때만 열림.
- **judge(심판):** 두 모델이 싸울 때 불러오는 3번째 모델.
- **consensus(합의):** 심판의 판정을 4번째 모델이 재확인해주는 구조.

왜 조건을 이렇게 까다롭게 걸었나? 심판(s99) 하나만으로는 "심판도 틀릴" 위험이 있습니다. 4번째 모델이 심판과 같은 답을 내면서 동시에 원래 후보 중 하나여야 한다는 이중 조건을 걸면, 완전히 엉뚱한 답으로 끌려갈 위험 없이 확신도만 높일 수 있습니다.

점수 상승 계단 (발표에서 그대로 말할 숫자):

```
0.79596  기본 2모델 앙상블
0.79629  + judge(s99) 게이트      (+0.00033)
0.79667  + consensus(s123) 게이트 (+0.00038)
0.79677  + consensus 가중치 0.10→0.15 (+0.00010) ← 최종
```

부수 효과 (반드시 언급): 추가 모델 2개가 5%의 행에서만 돌기 때문에, 5개 모델을 다 돌리는 것보다 서버 추론 시간을 크게 절약합니다. 10분 제한이 있는 이 대회에서 이건 성능이 아니라 가능/불가능의 문제였습니다.

⑦ 폴백(fallback) 설계

- 정의: 뭔가 실패해도 전체가 무너지지 않고 남은 것으로 계속 가는 설계.
- 우리 구현:

- 트랜스포머 하나가 로드 실패 → 그 모델만 빼고 남은 가중치로 자동 재정규화
- 시간 마감(deadline) 도달 → 처리 못 한 행은 **LightGBM 예측을 그대로 유지**
- 게이트 모델이 아예 못 돌면 → s7의 가중치가 0.25에서 **0.45로 자동 복귀** (`weight_when_gate_missing` 설정이 이것)
- **결과: 대회 기간 중 제출 실패 0회.**

가중치 재정규화란? 원래 $0.10+0.45+0.25=0.80$ 으로 나눠야 하는데 하나가 빠져서 $0.10+0.45=0.55$ 만 남으면, 0.55로 나눠서 다시 합이 1이 되게 맞추는 것. 이게 없으면 모델이 빠질 때마다 확률의 총량이 줄어 예측이 망가집니다.

⑧ 어휘 축소(vocabulary trimming) — 1GB 제약을 푼 열쇠

문제: mmBERT는 다국어라서 어휘가 약 25.6만 개. 모델 크기의 상당 부분이 **임베딩 테이블**(어휘 하나당 벡터 하나)입니다. 모델 하나가 **614MB**. 4개면 2.4GB → **1GB 제한 초과로 제출 불가.**

해결: 우리 데이터(7만 건 + 평가 입력 형식)에서 **실제로 등장하는 토큰은 전체의 3.8%뿐**. 나머지 96%는 아랍어·힌디어 등 우리가 절대 쓸 일 없는 토큰입니다. → 쓰는 토큰만 남기고 임베딩 테이블을 잘라냅니다.

- 모델당 **614MB → 273MB**
- **파리티(parity) 검증:** 자르기 전후의 예측이 **100% 동일**한지 전수 확인. (`trim_vocab.py`가 이 검증을 자동 수행) — "성능 손실 없음"을 실증한 것이 핵심.
- **remap 배열:** 원래 토큰 ID를 축소된 ID로 바꿔주는 대응표. 추론 시 이걸로 변환.

함께 쓴 기법:

- **FP16 (반정밀도):** 숫자를 32비트 대신 16비트로 저장. **용량 절반**, T4 GPU에서는 오히려 더 빠름. (정밀도 손실은 분류 성능에 무영향)
- **길이 버킷팅(bucketing):** 짧은 문장끼리 묶어서 배치를 만드는 것.
- **왜 빨라지나?** 배치 안에서는 가장 긴 문장에 맞춰 나머지를 **패딩(빈 칸 채우기)** 합니다. 10토큰과 500토큰을 같이 묶으면 10토큰짜리도 500토큰만큼 계산하게 됩니다. 길이가 비슷한 것끼리 묶으면 낭비가 사라집니다.
- 추가로 길이를 **32의 배수로 반올림** → 배치 모양의 종류가 줄어 GPU 커널 재준비 오버헤드 제거
- **결과: 추론 4.2배 가속**
- **최종:** 트랜스포머 4개 + LightGBM = **885MB ≤ 1GB** ✓

⑨ 데이터 누수(leakage)와 우리의 검증 ★

- **정의:** 실제로는 알 수 없어야 할 정보가 모델에 흘러들어가 점수가 부풀려지는 현상.

우리가 발견한 것: id가 `sess_...-step_02` 구조 → 같은 세션의 여러 **step**이 학습 데이터에 함께 들어 있습니다. 그런데 `step_05`의 `history`에는 **step_02에서 한 행동(= step_02의 정답)**이 그대로 기록되어 있습니다. 즉 다른 샘플을 조회하면 정답을 그냥 읽을 수 있었습니다.

- train 기준 **커버리지 86.6%, 조회 정확도 100%**

우리의 대응이 발표의 하이라이트: "쓰자/버리자"로 논쟁하지 않고 **제출 실험으로 검증**했습니다. 조회 로직을 넣은 제출과 뺀 제출의 리더보드 점수가 **소수점 10자리까지 동일**했습니다. → 평가 데이터는 세션당 1개 step만

출제되어 누수가 원천 차단되어 있음을 실증.

- **규정 관점:** 이 로직은 서버가 준 test 입력 **내부의 history**만 봅니다. 외부 정보 일절 없음. 평가 데이터에서 **발동 0건**을 실증했고, 학습에는 쓰지 않았습니다. 즉 **작동하지 않는 안전판**으로만 최종 모델에 남아 있습니다.

⑩ 세션 분리 검증 (GroupShuffleSplit)

- **문제:** 학습/검증을 무작위로 나누면 **같은 세션의 step들이 양쪽에 흩어집니다**. 검증 세트의 문제와 거의 같은 문제를 학습 때 이미 본 셈이라 점수가 **부풀려집니다**.
- **해결:** `GroupShuffleSplit` — **세션 ID를 그룹으로 묶어**, 한 세션은 통째로 학습이나 검증 중 한쪽에만 들어가게 분할.
- 우리 설정: **seed 42 고정, 검증 14,106건** — 전 실험을 **동일한 분할**로 비교.

왜 분할을 고정해야 하나? 분할이 바뀌면 점수 차이가 모델 때문인지 분할 운 때문인지 알 수 없습니다. 비교의 전제 조건입니다.

⑪ 검증 → 리더보드 매핑

초기 4번의 제출에서 **리더보드 = 검증 점수 + 약 0.005** 관계가 일관되게 나왔습니다. (검증 0.735 → LB 0.7414 등)

이게 왜 중요한가: "로컬 검증을 믿어도 된다"는 **근거를 데이터로 확보**한 것입니다. 그 뒤로는 제출하지 않고도 채택/기각을 결정할 수 있어 **실험 속도가 몇 배로 빨라**졌습니다.

다만 만능은 아니었습니다 → 그래서 다음 개념이 나옵니다.

⑫ LB 프로빙 (leaderboard probing)

- **정의:** 로컬 검증으로는 구분이 안 되는 **미세한 축을**, 소량의 실제 제출로 직접 재보는 것.
- 우리 원칙 (Q&A 대비 핵심):
- 차이가 0.003보다 크면 → 로컬 검증을 믿는다
- 차이가 0.002 이하면 → 검증 노이즈이므로 LB 프로브로 정점을 찾는다
- 이 방식으로 확인한 값들: LightGBM 가중치 **0.10** (0.07과 0.13은 양쪽 다 하락으로 실측), judge 가중치 **0.20** (0.15-0.25보다 우수), consensus 가중치 **0.15**, 온도 **0.28**.

Part 3. 우리 시스템이 실제로 돌아가는 순서

발표 슬라이드 11의 내용을, 서버에서 벌어지는 일 순서대로 풀어 쓴 것입니다.

[0] 서버가 zip을 풀고 `python script.py` 실행. 타이머 시작 (10분 제한)

[1] `test.jsonl` 30,000건 로드

[2] LightGBM 파이프라인

- └ TF-IDF로 텍스트 → 14개 클래스 확률 (스태킹 입력)
- └ 구조 피쳐 49개 + 표적 피쳐 11개 + ID 피쳐 13개 계산
- └ LightGBM 예측 → 30,000 × 14 확률 행렬
(이 시점에 이미 "최소한의 답"은 확보됨 = 폴백 기준선)

[3] 트랜스포머 s77 (v2 직렬화, 가중치 0.45)

- └ 각 샘플을 v2 방식으로 직렬화
- └ 길이순 정렬 → 32배수 버킷 배치 → FP16으로 GPU 추론
- └ 시간 초과 시 처리한 데이터까지만 `covered=True`

[4] 트랜스포머 s7 (v3 직렬화, 가중치 0.25) - 동일 절차

[5] ★게이트 판정★

`gate_mask = (s77 처리됨) & (s7 처리됨) & (s77의 답 ≠ s7의 답)`
→ 해당하는 행 인덱스만 추출 (약 5%, 1,500건 수준)

[6] 트랜스포머 s99 (judge, 가중치 0.20) - [5]에서 뽑힌 행만 추론

[7] 트랜스포머 s123 (consensus, 가중치 0.15) - 같은 행만 추론 후

`consensus 조건 = (s123답 == s99답) & (s123답 == s77답 or s123답 == s7답)`
조건 통과한 행만 `covered`로 남김

[8] 결합

분자 = $0.10 \times \text{LGBM} + \sum(\text{가중치} \times \text{각 모델 확률})$ ← 처리된 모델만
분모 = $0.10 + \sum(\text{가중치})$ ← 처리된 모델만
최종확률 = 분자 / 분모 ← 행마다 자동 재정규화

[9] `argmax` → 14개 클래스명 중 하나로 변환

[10] 세션 조회 오버라이드 (평가 데이터에선 0건 발동, 안전판)

[11] 클래스명 유효성 검사 → `output/submission.csv` 저장

각 단계에서 "실패해도 되는" 이유: [3]~[7] 어느 것이 실패하든 [8]의 분모/분자가 자동으로 조정되고, 최악의 경우 [2]의 LightGBM 예측만으로도 답이 나옵니다. **제출 실패가 구조적으로 불가능한 설계**입니다.

Part 4. 우리가 실패한 것들 (슬라이드 13 — 신뢰성 어필 구간)

발표에서 실패를 말하는 건 약점 고백이 아니라 "우리는 검증 체계로 이것 잡아냈다"는 강점 증명입니다.

시도	무엇을 한 것인가	결과	교훈
클래스별 scale 후처리	클래스마다 확률에 배수를 곱해 Macro-F1을 올리려 함	검증은 ↑, LB는 ↓ → 즉시 제거하니 +0.0013 회복	검증 과적합은 LB 프로브로만 검출 가능
모델 수프	시드 다른 모델들의 가중치를 직접 평균	0.7645로 붕괴	시드가 다르면 다른 basin — 평균 불가
3-way + XLM-R	구형 백본을 3번째 모델로 추가	LB 하락 (0.79104)	약한 모델을 넣으면 강한 신호가 희석됨
전수 탐색 (41만 조합)	작은 부분집합(1,400건)에서 모든 조합을 다 시도해 최고를 고름	로컬 0.812 → LB 0.795	소표본 조합 탐색 = 강한 과적합. 로컬-LB 상관 0.20 실측
학습형 메타 게이트	"어떤 모델을 믿을지"를 또 다른 모델로 학습	LB -0.005	트랜스포머 확률이 전체 학습본에서 나와 OOF에도 누수 잔존

여기서 나온 운영 원칙 (발표 마지막 강조 문장):

리더보드에서 검증된 코어(s77+s7)는 절대 바꾸지 않는다. 새로운 시도는 언제나 champion을 잠근 채 상방 베팅으로만 제출한다.

이 원칙 덕분에, 마감 직전 여러 실험이 실패했어도 **최고점은 한 번도 잃지 않았습니다.**

Part 5. 슬라이드별 "무엇을 이해하고 올라가야 하는가"

각 슬라이드마다 ① 핵심 메시지 ② 반드시 이해해야 할 개념 ③ 여기서 나올 법한 질문.

슬라이드 1 — 표지

- **메시지:** 0.43 → 0.797. 이 +0.37이 어디서 왔는지를 심사 항목 순서로 설명하겠다.
- **이해:** 0.43은 검증 기준 베이스라인, 0.797은 리더보드 기준 최종. 기준이 다르다는 걸 알고 말할 것.

슬라이드 2 — 문제 정의

- **메시지:** 제약(Macro-F1, 10분, 1GB, 코드 제출)이 곧 설계 조건이었다.
- **이해:** Part 1-3, 1-4를 확실히. "왜 Macro-F1이 소수 클래스를 중요하게 만드는가"를 자기 말로 설명할 수 있어야 함.

슬라이드 3 — 데이터 분석 ① 신호의 발견

- **메시지:** 베이스라인이 버리던 신호를 정량으로 찾아냈고, 실험으로 +0.17을 확인했다.
- **핵심 숫자:** write_file→edit_file **40%**, 직전 행동 단독 정확도 **23%**, 첫 턴의 plan_task **12%** (전체 3.8%의 3배), prompt만 **0.437** → 이력+메타 추가 **0.611**
- **예상 질문:** "직전 행동 정확도가 23%밖에 안 되는데 왜 중요한가?" → 단독으로는 약하지만 텍스트와 결합했을 때 +0.17을 만들었습니다. 두 신호가 보완적입니다.

슬라이드 4 — 누수 검증

- **메시지:** 누수 후보를 발견했고, 버리지도 쓰지도 않고 제출 실험으로 검증했다.
- **이해:** Part 2-⑨ 전체.
- **예상 질문:** (Part 6에 상세)

슬라이드 5 — 라벨 모호성

- **메시지:** 같은 발화에 다른 정답이 달린 경우가 많아, 텍스트만으로는 정확도 상한 약 75%.
- **왜 이 분석을 했나:** 이게 후반 전략을 "더 큰 모델"이 아니라 "모호 구간의 마진 싸움"으로 바꾼 근거입니다. 리더보드 상위권이 0.79~0.80에 밀집한 것도 이 분석과 일치합니다.
- **예상 질문:** "상한이 75%인데 어떻게 F1 0.797이 나오나?" → 그 75%는 중복 발화 그룹(전체의 약 15%)에 한정된 상한이고, 지표도 정확도입니다. 나머지 85%는 이력·메타로 구분 가능해서 전체 Macro-F1은 더 높게 나옵니다.

슬라이드 6 — 검증 체계

- **메시지:** 네 축(세션 분리 / 검증-LB 매핑 / LB 프로빙 / 과적합 조기 검출).
- **이해:** Part 2-⑩ ⑪ ⑫ + Part 4.

슬라이드 7 — 검증이 만든 의사결정

- 핵심 숫자: 100% 재학습 프리미엄 **+0.004~0.007**, 이견율 **7.3% vs 13.5%**, mmBERT **+0.021**
- "100% 재학습 프리미엄"이란: 실험할 땐 80%로 학습해 20%로 검증하지만, **제출용은 100% 데이터로 다시 학습**합니다. 학습 데이터가 20% 더 많으니 그만큼 좋아지고, 그 차이를 실측한 값입니다.

슬라이드 8 — 발전 단계

- 메시지: 8단계 누적 개선. 후반 도약은 **ID토큰(+0.011)**과 **게이트 앙상블**.
- 표를 통째로 외우려 하지 말고, Part 0-3의 궤적 6줄만 기억하면 충분합니다.

슬라이드 9 — ID토큰 직렬화 ★

- 이해: Part 2-② 전체. 여기서 절대 서두르지 말 것.
- 핵심 논리 3단: LGBM 최강 피처가 세션번호였다 → 트랜스포머는 못 보고 있었다 → 토큰으로 쪼개 주입했더니 **+0.011**.

슬라이드 10 — 게이트 앙상블 ★

- 이해: Part 2-⑥ 전체. **Part 0-2의 위원회 비유로 설명하면 가장 잘 전달**됩니다.
- 계단 숫자: 0.79596 → 0.79629 → 0.79667 → 0.79677

슬라이드 11 — 최종 아키텍처

- 가중치 암기: LGBM **0.10** / s77 **0.45** / s7 **0.25**(게이트 미작동 시 0.45) / s99 **0.20** / s123 **0.15**, 온도 **0.28**
- 강조: 폴백 설계 → 제출 실패 **0회**

슬라이드 12 — 제약 엔지니어링

- 이해: Part 2-⑧ 전체.
- 핵심 숫자: 어휘 **3.8%**만 유지, **614MB → 273MB**, 파리티 **100%**, 버킷팅 **4.2배** 가속, 최종 **885MB**

슬라이드 13 — 실패에서 배운 것

- 이해: Part 4 전체. 마무리 문장(champion 잠금 + 상방 베팅)을 꼭 살릴 것.

슬라이드 14 — 마무리

- 한 문장: 데이터 분석이 방향을 정했고, 검증 체계가 속도를 만들었고, 알고리즘이 마진을 쌓았다.
-

Part 6. 예상 질문과 답변 (Q&A 대비)

A. 반드시 나올 질문

Q1. 세션 조회 오버라이드는 데이터 누수 아닌가요? 규정 위반 소지는?

세 가지로 답변드리겠습니다. 첫째, 이 로직은 평가 서버가 제공한 test 입력 내부의 history만 참조합니다. 외부 데이터나 외부 정보는 일절 사용하지 않습니다. 둘째, 실제로 발동하지 않습니다. 조회 로직을 넣은 제출과 뺀 제출의 리더보드 점수가 소수점 10자리까지 동일했고, 이로써 평가 데이터는 세션당 1개 step만 출제되어 커버리지가 0임을 실증했습니다. 셋째, 학습에는 전혀 사용하지 않았습니다. 즉 커버 0이면 무영향인 안전판으로만 남아 있습니다.

Q2. 리더보드에 맞춰 튜닝한 건 Public 과적합 아닌가요?

이 대회는 **private 분할이 없습니다**. Public Score가 전체 테스트 데이터 100% 채점이고, 예선 마감 시점의 Public이 그대로 Private가 됩니다. 따라서 리더보드에 맞추는 것은 과적합이 아니라 **실제 목적함수를 직접 최적화하는 것**입니다. 다만 무분별하게 쓰지 않았습니다. 차이가 0.003을 넘는 큰 축은 로컬 검증으로 결정하고, 검증 노이즈 수준인 0.002 이하 미세 축만 제한된 제출 슬롯으로 확인했습니다.

Q3. LightGBM 가중치가 0.10뿐이면 왜 넣나요?

두 가지 이유입니다. 첫째, ID토큰 직렬화 이후 트랜스포머가 LightGBM의 구조 신호를 흡수해서 중복이 커졌고, LB 프로브로 0.10이 정점임을 실측했습니다. 0.07과 0.13은 양쪽 다 하락했습니다. 둘째, **폴백 기준선 역할**입니다. 트랜스포머가 실패하거나 10분 마감에 걸린 행은 LightGBM 예측이 그대로 나갑니다. 가중치는 0.10이지만 안정성 측면에서는 필수 구성입니다.

Q4. 게이트 임계값은 어떻게 정했나요?

이건 여부는 **임계값이 아니라 구조적 조건**입니다. 두 모델의 argmax가 다른가 같은가라는 이진 판정이라 튜닝할 하이퍼파라미터가 없습니다. 튜닝 대상은 judge와 consensus의 **가중치**였고, LB 프로브로 각각 0.20과 0.15가 정점임을 확인했습니다. judge는 0.15와 0.25 양쪽이 모두 낮았고, consensus는 0.10에서 0.15로 올릴 때 개선, 하드 오버라이드(가중치 무한대)는 하락이었습니다.

Q5. 검증 점수와 리더보드가 다른데 뭘 믿었나요?

축마다 신뢰 도구를 달리했습니다. 구조적 변경처럼 차이가 큰 축(>0.003)은 로컬 검증이 정확했고, 실제로 초기 4연속 제출에서 LB = 검증 + 0.005 관계가 일관되게 나와 근거를 확보했습니다. 반면 가중치·온도 같은 후처리 축은 검증 차이가 노이즈 수준이라 소량 프로브 제출로만 정점을 찾았습니다.

B. 기술 심화 질문

Q6. 왜 시드 앙상블만으로는 부족했나요?

실측했습니다. 같은 백본·같은 직렬화에서 시드만 바꾼 모델끼리는 **이건율이 7.3%**에 그쳤습니다. 반면 직렬화나 아키텍처가 다르면 **13.5%**까지 올라갑니다. 앙상블의 이득은 모델이 서로 다른 실수를 할 때 나오므로, **다양성은 시드가 아니라 표현과 아키텍처에서 만들어야 한다**는 결론을 데이터로 확정했습니다.

Q7. 왜 더 큰 모델(large 계열)을 쓰지 않았나요?

두 가지 제약 때문입니다. 첫째, **1GB 용량 제한**입니다. base 모델도 어휘 축소 없이는 614MB라 4개를 담을 수 없었습니다. 둘째, **10분 추론 제한**입니다. 그리고 더 근본적으로, 라벨 모호성 분석에서 **텍스트 신호의 상한이 낮다**는 걸 확인했기 때문에 모델 크기를 키우는 것보다 표현 다양성과 앙상블 구조에 투자하는 게 기대 이득이 크다고 판단했습니다.

Q8. 온도 0.28은 무슨 의미인가요?

각 모델의 확률에 0.28 제곱을 하고 다시 정규화합니다. 결과적으로 **확률 분포가 평평해집니다**. 개별 모델이 지나치게 확신에 차 있으면 앙상블에서 그 모델이 독주해서 다른 모델의 2순위·3순위 의견이 반영되지 않습니다. 평평하게 만들어 여러 모델의 의견이 실제로 섞이도록 한 것이고, 0.28은 LB 프로브로 찾은 정점입니다.

Q9. 어휘를 3.8%만 남기면 성능이 떨어지지 않나요?

떨어지지 않는 것을 **검증했습니다**. 축소 전후 모델의 예측이 100% 동일한지를 전수 비교하는 파티티 검증을 trim_vocab.py 에 내장했고, 완전 일치를 확인한 뒤에만 제출에 사용했습니다. 원리적으로도 손실이 없습니다. 잘라낸 토큰은 **우리 데이터에 한 번도 등장하지 않는** 다른 언어의 토큰이라, 추론 시 조회될 일 자체가 없습니다.

Q10. 재현 가능한가요?

가능합니다. LightGBM은 random_state 42, 트랜스포머는 시드 7·77·99·123으로 고정했고 torch·numpy·python random 시드를 스크립트 내에서 모두 고정했습니다. 라이브러리 버전도 전부 명시했습니다. 특히 **학습 코드가 추론 코드에서 피쳐 함수를 직접 import** 하는 구조라, 학습과 추론 사이의 피쳐 불일치가 원천적으로 불가능합니다. 다만 GPU 부동소수점 연산의 비결정성으로 소수점 미세 차이는 발생할 수 있으며, 이는 대회 규정에서 허용하는 범위입니다.

Q11. 외부 데이터를 쓰지 않았나요?

학습에는 **대회 제공 train 70,000건만** 사용했습니다. 사전학습 백본(jhu-clsp/mmBERT-base)은 공개 가중치이며, 규정상 라이브러리·모델 사용은 자유이고, 오프라인 평가 환경이라 공식 가이드의 권장대로 zip에 포함했습니다.

Q12. 게이트가 5%에만 작동한다면 개선 폭이 작은 것 아닌가요?

맞습니다. 그리고 그게 **의도한 설계**입니다. 이 시점에서 저희는 이미 0.796 구간이었고, 상위권이 0.79~0.80에 밀집해 있었습니다. 남은 개선 여지는 **두 모델이 갈리는 어려운 구간**에만 있었습니다. 나머지 95%는 이미 두 모델이 합의하는, 즉 대부분 맞히고 있는 구간이라 건드릴 이유가 없었습니다. 게이트는 **개선 여지가 있는 곳에만 계산 자원을 집중**하는 구조이고, 실제로 0.79596에서 0.79677까지 계단식 상승을 만들었습니다.

C. 곤란할 수 있는 질문

Q13. 0.797과 1등 0.798의 차이는 사실상 없는 것 아닌가요?

절대 차이는 작습니다. 다만 저희는 그 구간이 **라벨 모호성으로 인한 천장 근처**라는 걸 데이터로 확인했고, 그래서 후반 전략을 성능 경쟁이 아니라 **모호 구간에서의 마진 확보**로 전환했습니다. 이 판단 자체가 데이터 분석의 결과이고, 저희가 보여드리고 싶은 부분입니다.

Q14. 실패한 실험이 많은데 시간 낭비 아니었나요?

실패한 실험 5건은 모두 **검증 체계가 조기에 잡아낸 것**이고, 만약 그 체계가 없었다면 그중 하나를 최종 제출로 선택했을 겁니다. 예를 들어 전수 탐색 결과는 로컬에서 0.812로 저희 champion보다 훨씬 높아 보였지만 실제 LB는 0.795였습니다. **실패를 기록하고 기각한 것이 최종 점수를 지켰습니다.**

Q15. 이 모델을 실제 서비스에 쓸 수 있나요?

그대로는 어렵습니다. 이 모델은 대회 데이터 분포와 14개 도구 행동 클래스에 특화되어 있고, 특히 **ID토큰은 이 데이터 생성 방식에 의존하는 피처**라 실제 서비스에서는 일반화되지 않습니다. 다만 **게이트 앙상블 구조와 풀백·시간예산 설계는 도메인과 무관하게 재사용** 가능합니다. 어려운 입력에만 큰 모델을 호출하는 구조는 실서비스의 비용 최적화에 바로 적용됩니다.

Part 7. 숫자 암기 카드

발표 중 실제로 입에 올릴 숫자만 추렸습니다. 붉은 것은 반드시 암기.

결과

- 최종 Private Macro-F1 **0.79677** (정확히는 0.7967670316)
- 베이스라인 검증 **0.4309** → 개선폭 약 **+0.37**
- 1등 0.79795 / 우리와의 격차 약 0.0012

문제·데이터

- 클래스 **14개** / 학습 **70,000건** / 평가 **30,000건**
- 클래스 불균형 **9배** (edit_file 11,171 vs web_search 1,273)
- 검증 세트 14,106건 (GroupShuffleSplit seed 42)

데이터 분석

- prompt만 **0.437** → +이력·메타 토큰 **0.611** (+**0.17**)
- write_file → edit_file **40%**, read_file → edit_file 29%
- 직전 행동 단독 정확도 **23%**
- 첫 턴 plan_task **12%** (전체 3.8%의 3배)
- 누수 커버리지 **86.6%**, 조회 정확도 **100%**, 평가 데이터 커버 **0%**
- 중복 발화 그룹 약 15%에서 정확도 상한 약 **75%**

검증

- 검증 → LB 매핑: **+0.005** 일관
- 이견율: 같은 계열 **7.3%** / 이종 **13.5%**
- 100% 재학습 프리미엄 **+0.004 ~ +0.007**
- 전수 탐색: 로컬 **0.812** → LB **0.795**, 상관 **0.20**

알고리즘

- ID토큰 직렬화 **+0.011** (단일 기법 최대)
- mmBERT 교체 **+0.021** (단일 백본 변경 최대)
- 게이트 계단: **0.79596** → **0.79629** → **0.79667** → **0.79677**
- 게이트 발동률 약 **5%**
- 가중치: LGBM **0.10** / s77 **0.45** / s7 **0.25** / s99 **0.20** / s123 **0.15**

- 온도 **0.28**, max_len **512**, 3에폭, batch 16, lr 3e-5

엔지니어링

- 어휘 유지 **3.8%** / 모델당 **614MB** → **273MB** / 파리티 **100%**
 - 버킷팅 **4.2배** 가속 / 최종 zip **885MB** (제한 1GB)
 - 서버: T4 16GB, 3 vCPU, **10분**, 인터넷 차단
 - 제출 실패 **0회**
-

Part 8. 자가 점검 퀴즈

답을 보지 말고 소리 내어 답해 보세요. 막히는 항목이 그날 공부할 부분입니다.

기초

1. Macro-F1이 일반 정확도와 다른 점은? 그게 우리 전략에 어떤 영향을 줬나?
2. 이 대회가 CSV가 아니라 코드를 제출하게 하는 이유는?
3. 파인튜닝과 사전학습의 차이를 한 문장으로.
4. 왜 검증 데이터를 무작위가 아니라 세션 단위로 나눴나?

핵심 기법

5. "직렬화"란 무엇이고, 왜 필요한가?
6. ID토큰이란 무엇이며, 왜 세션 번호를 그대로 안 넣고 나머지 연산을 했나?
7. v2와 v3 직렬화의 차이는? 왜 둘 다 쓰는가?
8. 게이트 앙상블에서 3번째 모델은 언제 호출되나? 4번째 모델의 조건 두 가지는?
9. 온도 0.28은 확률 분포를 어떻게 바꾸며, 왜 그게 앙상블에 유리한가?
10. 어휘 축소로 용량이 줄어드는 원리는? 성능 손실이 없다는 걸 어떻게 증명했나?
11. 길이 버킷팅이 추론을 빠르게 하는 원리는?
12. 폴백 설계에서 "재정규화"가 없으면 무슨 일이 벌어지나?

판단·설계

13. 모델 수프가 실패한 이유를 basin 개념으로 설명하라.
14. 전수 탐색이 로컬 0.812인데 LB 0.795였다. 무슨 일이 일어난 건가?
15. 우리는 언제 로컬 검증을 믿고, 언제 LB 프로브를 썼나? 기준 수치는?
16. 누수를 발견하고 "쓰지 않기"가 아니라 "실험으로 검증"을 택한 이유는?
17. LightGBM 가중치가 0.10인데도 파이프라인에서 뺄 수 없는 이유는?

답 확인 (막힌 것만 펼쳐 보기)

1. 14개 클래스의 F1을 동일 가중으로 평균 → 1,273건짜리 web_search가 11,171건짜리 edit_file과 같은 무게. 소수 클래스 관리가 순위를 가름. → Part 1-3
2. 평가 데이터를 비공개로 유지해 치팅·평가 데이터 과적합을 차단하고, 정확도뿐 아니라 속도·메모리·안정성까지 실배포와 유사한 조건에서 측정하기 위해. → Part 1-4
3. 사전학습은 방대한 텍스트로 언어 자체를 배우는 것, 파인튜닝은 그 모델을 우리 데이터로 추가 학습해 우리 문제 전용으로 만드는 것. → Part 2 레벨2
4. 같은 세션의 여러 step이 학습·검증에 갈리면 거의 같은 문제를 이미 본 셈이라 점수가 부풀려짐. GroupShuffleSplit으로 세션을 통째로 한쪽에만 배치. → Part 2-⑩
5. 프롬프트·이력·메타를 트랜스포머가 먹을 수 있는 하나의 긴 문자열로 합치는 것. 트랜스포머는 텍스트 한 덩어리만 입력받기 때문. → Part 2-①
6. 세션 일련번호를 토큰화해 트랜스포머 입력에 주입한 것. 큰 정수는 토큰라이저가 이상하게 자르고 모델이 잘 다루지 못해서, 256·100·64·16 나머지로 쪼개 작은 정수 여러 개로 노출. → Part 2-②
7. $v_2 = \text{ID토큰} + \text{기본}$, $v_3 = \text{ID토큰} + \text{구조 신호 토큰} + \text{기본}$. 성능은 비슷한데 **틀리는 문제가 다르기** 때문에 앙상블 다양성의 재료가 됨. → Part 2-③
8. 기본 2모델(s77, s7)의 argmax가 다를 때(약 5%) 3번째(s99) 호출. 4번째(s123)는 ① s99와 같은 답이고 ② 그 답이 기본 2모델 중 한쪽의 답과도 일치할 때만 반영. → Part 2-⑥
9. 확률에 0.28 제공 후 재정규화 → 분포가 평평해짐. 한 모델의 과도한 확신을 눌러 다른 모델의 2·3순위 의견까지 섞이게 함. → Part 2 레벨2 온도
10. 다국어 어휘 25.6만 개 중 실제 등장은 3.8% → 임베딩 테이블을 잘라냄(614→273MB). 축소 전후 예측 100% 일치(파리티) 전수 검증으로 증명. → Part 2-⑧
11. 배치 안에서는 가장 긴 문장에 맞춰 패딩하므로, 길이가 비슷한 것끼리 묶으면 낭비되는 계산이 사라짐. 32배수 반올림으로 배치 shape 종류도 줄임 → 4.2배. → Part 2-⑧
12. 모델이 빠지면 확률의 총량이 줄어들어 분포가 망가짐. 남은 가중치의 합으로 나눠 다시 합이 1이 되게 맞춰야 함. → Part 2-⑦
13. 시드가 다르면 학습이 서로 다른 골짜기(basin)로 수렴하고 특히 분류 헤드의 좌표계가 달라짐. 두 지점의 중간은 어느 쪽도 아닌 무의미한 지점 → 0.7645로 붕괴. → Part 2-⑤
14. 1,400건 소표본에서 41만 조합을 다 시도해 최고를 골랐으므로, 그 표본의 우연한 노이즈에 맞춘 조합이 선택됨 = 강한 과적합. 로컬-LB 상관 0.20 실측. → Part 4
15. 차이 0.003 초과 = 구조적 변경 → 로컬 검증. 0.002 이하 = 검증 노이즈 → LB 프로브. → Part 2-⑫
16. 쓰면 규정·일반화 위험, 버리면 정말 이득인지 알 수 없음. 넣은 제출과 뺀 제출의 점수를 비교해 **평가 데이터에서 발동 0건**임을 실증 → 안전판으로만 남김. → Part 2-⑨
17. 트랜스포머가 실패하거나 10분 마감에 걸린 행의 **풀백 기준선**이기 때문. 가중치는 낮지만 안정성 측면에서 필수. → Part 6 Q3

Part 9. 발표 직전 체크리스트

전날

- ☐ Part 0-2(위원회 비유)를 아무것도 안 보고 설명할 수 있다
- ☐ Part 7 숫자 카드 중 굵은 항목을 다 말할 수 있다
- ☐ Part 8 퀴즈 17문항 중 막히는 게 3개 이하다
- ☐ 슬라이드 9:10을 각각 60초로 연습했다 (여기가 알고리즘 점수의 핵심)

당일

- ☐ 숫자는 세 자리까지만 읽기 — "영점칠구칠", "플러스 영점영일일" (단 게이트 계단 $0.79596 \rightarrow 0.79677$ 은 예외적으로 그대로)
- ☐ 슬라이드 9:10에서는 **절대 서두르지 않기**
- ☐ 슬라이드 4(누수) 후 심사위원 표정에 의문이 보이면, Q&A에서 선제적으로 "규정 관련 백업 슬라이드가 있습니다"라고 언급
- ☐ 시간 초과 시 삭감 순서: 슬라이드 4의 규정 문장 $\rightarrow$ 슬라이드 7 압축 $\rightarrow$ 슬라이드 12 압축

모르는 질문을 받았을 때

"그 부분은 저희가 실험으로 확인하지 않은 영역입니다. 다만 관련해서 확인한 것은 ○○이고, 그 근거로는 △△를 실측했습니다."

모르는 걸 아는 척하지 말 것. 이 팀의 강점은 "전부 실험으로 확인했다"는 태도이고, 확인 안 한 걸 확인한 것처럼 말하는 순간 그 강점이 무너집니다.

관련 문서

- 발표 슬라이드 구성 — 슬라이드에 그대로 옮길 표·불릿
- 발표 대본 — 10분 시간 배분 포함 실제 발화 대본
- 대회 정리 — 대회 규정·데이터 구조 원본 정리
- 대회 전략 전체 기록 — 모든 실험의 날짜별 로그
- 학습 코드 README — 재현 방법과 환경
- 모델 카드 — 최종 모델 요약